

Comparaison de RSA et ElGamal dans le cadre du chiffrement léger

Document de révision — préparation à la soutenance

Basile Capier — 45879

Table des matières

1	Vocabulaire de la cryptographie	2
1.1	Symétrique vs asymétrique	2
1.2	Chiffrement léger	2
2	RSA en profondeur	2
2.1	Algorithme complet	2
2.2	Preuve de correction	3
2.3	Sécurité	3
2.4	Optimisation : CRT	3
3	ElGamal en profondeur	3
3.1	Approche : exploiter la cyclicité par un sous-groupe d'ordre premier	3
3.2	Génération des paramètres	3
3.3	Algorithme complet	4
3.4	Sécurité	4
3.5	Sécurité sémantique	4
3.6	Faible du k réutilisé	4
4	Structures algébriques sous-jacentes	4
4.1	La «non-cyclicité» de RSA : un mythe à dissiper	4
4.2	Théorème chinois des restes	5
4.3	Square-and-multiply	5
4.4	Récapitulatif des coûts	5
5	Protocole Python détaillé	5
5.1	Bibliothèques	5
5.2	Code RSA	5
5.3	Code ElGamal optimisé	5
5.4	Docker	6
6	Résultats Python	6
6.1	Pleine puissance	6
6.2	Sous contrainte Docker	6
6.3	Pourquoi le ratio n'est pas monotone ?	6
6.4	Le chiffrement	7

7	JavaCard : architecture et implémentation	7
7.1	Architecture	7
7.2	APDU	7
7.3	Mon applet RSA	7
7.4	Mon applet ElGamal	7
7.5	Pourquoi ElGamal reste lent	7
8	Résultats JavaCard	8
8.1	Mesures (une exécution)	8
8.2	Interprétation	8
8.3	Cohérence avec Python	8
9	Synthèse et conclusion	8
10	Questions probables du jury et éléments de réponse	8
11	Valeurs à connaître par cœur	11
12	Limites à mentionner honnêtement	11

1 Vocabulaire de la cryptographie

1.1 Symétrique vs asymétrique

Cryptographie symétrique. Une seule clé secrète, partagée entre les deux parties. Rapide mais pose le problème de l'échange de clé. *Exemple* : AES.

Cryptographie asymétrique. Paire de clés : publique (diffusable), privée (gardée secrète). Plus lente, mais résout l'échange de clés. *Exemples* : RSA, ElGamal, ECC.

Usage hybride en pratique. On utilise l'asymétrie pour échanger une clé de session, puis le symétrique pour chiffrer les données (TLS, PGP, etc.).

1.2 Chiffrement léger

Ensemble des primitives cryptographiques conçues pour ou adaptées à des environnements à ressources contraintes (RAM, CPU, énergie). Cibles : RFID, cartes à puce, IoT, dispositifs médicaux.

2 RSA en profondeur

2.1 Algorithme complet

Génération.

1. Tirer p, q premiers distincts ; $n = pq$.
2. $\varphi(n) = (p - 1)(q - 1)$.
3. Choisir e tel que $\text{gcd}(e, \varphi(n)) = 1$. *Standard* : $e = 65\,537 = 2^{16} + 1$.
4. $d \equiv e^{-1} \pmod{\varphi(n)}$ via Euclide étendu.

Clé publique : (n, e) . **Clé privée** : (n, d) .

Chiffrement. $c = m^e \bmod n$ (avec $0 \leq m < n$).

Déchiffrement. $m = c^d \bmod n$.

2.2 Preuve de correction

Résultat-clé

Théorème d'Euler. Si $\gcd(a, n) = 1$, alors $a^{\varphi(n)} \equiv 1 \pmod{n}$.

Correction RSA. $ed \equiv 1 \pmod{\varphi(n)}$ donc $\exists k$ tel que $ed = 1 + k\varphi(n)$. Donc

$$c^d = (m^e)^d = m^{ed} = m^{1+k\varphi(n)} = m \cdot (m^{\varphi(n)})^k \equiv m \pmod{n}.$$

2.3 Sécurité

Sécurité $\equiv$ difficulté de factoriser n . Algorithme connu le plus efficace : crible algébrique (GNFS), complexité *sous-exponentielle*.

Tailles ANSSI 2024 : RSA-2048 jusqu'en 2030 (112 bits de sécurité), RSA-3072 au-delà (128 bits).

2.4 Optimisation : CRT

En pratique, RSA utilise le théorème chinois des restes : on calcule $m_p = c^{d_p} \pmod{p}$ et $m_q = c^{d_q} \pmod{q}$ avec $d_p = d \pmod{p-1}$, $d_q = d \pmod{q-1}$, puis on reconstitue m . Gain : facteur $\sim 4\times$.

Attention — piège fréquent

Mon implémentation Python n'utilise pas le CRT. La JavaCard, elle, l'utilise systématiquement via le coprocesseur.

3 ElGamal en profondeur

3.1 Approche : exploiter la cyclicité par un sous-groupe d'ordre premier

ElGamal travaille traditionnellement dans le groupe multiplicatif $(\mathbb{Z}/p\mathbb{Z})^*$. Mais pour exploiter réellement la structure cyclique, on restreint les calculs à un **sous-groupe d'ordre premier q** , avec $|q| \ll |p|$. C'est l'approche standard du standard DSA (FIPS 186).

Le bénéfice : les exposants sont choisis dans $\{1, \dots, q-1\}$, donc de taille $|q|$ bits (typiquement 256) au lieu de $|p|$ bits (typiquement 2048+). Square-and-multiply coûte $\Theta(|\exp| \cdot M(|p|))$, donc on passe de $\Theta(|p|^3)$ à $\Theta(|q| \cdot |p|^2)$, soit un gain théorique de $|p|/|q|$ ($16\times$ pour $|p| = 4096$).

3.2 Génération des paramètres

1. Tirer q premier de $|q|$ bits.
2. Tirer h aléatoire jusqu'à ce que $p = hq + 1$ soit premier de taille voulue.
3. Tirer $\alpha \in \{2, \dots, p-2\}$; poser $g = \alpha^h \pmod{p}$. Si $g = 1$, recommencer.

Résultat-clé

g est d'ordre exactement q dans $(\mathbb{Z}/p\mathbb{Z})^*$.

Démonstration. Par Fermat, $\alpha^{p-1} \equiv 1 \pmod{p}$. Or $p-1 = hq$, donc

$$g^q = (\alpha^h)^q = \alpha^{hq} = \alpha^{p-1} \equiv 1 \pmod{p}.$$

Donc $\text{ord}(g) \mid q$. Comme q premier, $\text{ord}(g) \in \{1, q\}$. Le cas $\text{ord}(g) = 1$ est exclu ($g \neq 1$). Donc $\text{ord}(g) = q$. ■

3.3 Algorithme complet

Clés. $x \in \{1, \dots, q-1\}$ aléatoire (clé privée, courte). $y = g^x \bmod p$ (publique).

Chiffrement. Tirer $k \in \{1, \dots, q-1\}$ (éphémère, court). Calculer $c_1 = g^k$, $c_2 = m \cdot y^k$, tous mod p .

Déchiffrement. $s = c_1^x \bmod p$, $m = c_2 \cdot s^{-1} \bmod p$.

Correction.

$$c_2 \cdot (c_1^x)^{-1} = (m \cdot y^k)(g^{kx})^{-1} = m \cdot g^{xk} \cdot g^{-xk} = m \pmod{p}.$$

3.4 Sécurité

Sécurité $\equiv$ difficulté du *problème du logarithme discret* (DLP) dans le sous-groupe d'ordre q . Algorithmes connus :

- **Crible (Index Calculus)** dans $(\mathbb{Z}/p\mathbb{Z})^*$ entier : complexité sous-exponentielle en $|p|$. Pour 112 bits de sécurité : $|p| \geq 2048$.
- **Pollard- ρ** dans le sous-groupe d'ordre q : complexité $\sqrt{q}$. Pour 112 bits : $|q| \geq 224$.

La sécurité effective est le minimum. **Standard DSA** : $|p| = 2048$, $|q| = 256 \rightarrow 112$ bits.

3.5 Sécurité sémantique

ElGamal a une propriété qu'RSA déterministe n'a pas : deux chiffrements du même m donnent des résultats différents (grâce à l'aléa k). C'est la *sécurité sémantique*. En pratique, RSA utilise un padding (OAEP) pour pallier.

3.6 Faille du k réutilisé

Si deux chiffrements utilisent le même k : $c_2/c_2' = m/m'$. Connaître un message révèle l'autre. D'où l'importance critique du PRNG.

4 Structures algébriques sous-jacentes

4.1 La «non-cyclicité» de RSA : un mythe à dissiper

$(\mathbb{Z}/n\mathbb{Z})^*$ avec $n = pq$ est généralement non cyclique : par CRT, $(\mathbb{Z}/n\mathbb{Z})^* \cong (\mathbb{Z}/p\mathbb{Z})^* \times (\mathbb{Z}/q\mathbb{Z})^*$, et les ordres $p-1, q-1$ sont tous deux pairs (donc 2 divise leur pgcd).

Mais RSA n'exploite pas cette propriété. Il fonctionnerait dans n'importe quel groupe abélien fini d'ordre connu.

Attention — piège fréquent

Le MCOT initial parlait de «groupe cyclique vs non cyclique». Cette formulation est imprécise. Le jury de maths peut s'en saisir.

Bonne formulation : « ElGamal exploite la structure de groupe cyclique ; RSA s'appuie sur l'arithmétique modulaire dans un anneau quotient, sans recourir à la cyclicité de son groupe multiplicatif. »

4.2 Théorème chinois des restes

Résultat-clé

Si $\gcd(m_1, m_2) = 1$, $\mathbb{Z}/(m_1 m_2)\mathbb{Z} \cong \mathbb{Z}/m_1\mathbb{Z} \times \mathbb{Z}/m_2\mathbb{Z}$.

Applications : (i) RSA-CRT pour accélérer le déchiffrement, (ii) montrer que $(\mathbb{Z}/n\mathbb{Z})^*$ est non cyclique.

4.3 Square-and-multiply

Coût d' $a^b \bmod n$: $|b|$ carrés + au plus $|b|$ multiplications = $\Theta(|b| \cdot M(|n|))$.

Coût de $M(k)$: $\Theta(k^2)$ scolaire, $\Theta(k^{1.58})$ Karatsuba, $\Theta(k \log k \log \log k)$ FFT.

4.4 Récapitulatif des coûts

Opération	RSA	ElGamal (sous-groupe q)
Chiffrement	1 expo, $ e = 17$ bits	2 expo, $ k = q $ bits
Déchiffrement	1 expo, $ d \approx n $ bits	1 expo, $ x = q $ bits + inversion

5 Protocole Python détaillé

5.1 Bibliothèques

`pycryptodome` : `getPrime(k)` (Miller-Rabin), `isPrime`, `inverse.pow(a, b, n)` est natif Python et utilise GMP (square-and-multiply à fenêtre glissante).

`time.perf_counter()` : compteur haute résolution.

5.2 Code RSA

```
1 def my_rsa_generate_keys(key_size):
2     p, q = getPrime(key_size//2), getPrime(key_size//2)
3     n = p * q
4     phi_n = (p-1) * (q-1)
5     e = 65537
6     d = pow(e, -1, phi_n)
7     return (n, e), (n, d)
```

5.3 Code ElGamal optimisé

```
1 def generate_dsa_like_params(p_size, q_size):
2     q = getPrime(q_size)
3     h_size = p_size - q_size
4     while True:
5         h = secrets.randbits(h_size - 1) | (1 << (h_size - 1))
6         if h % 2 == 1: h += 1
7         p = h * q + 1
8         if p.bit_length() == p_size and isPrime(p):
9             break
10    while True:
11        alpha = random.randint(2, p - 2)
12        g = pow(alpha, h, p) # g d'ordre q
13        if g != 1: break
14    return p, q, g
15
16 def elgamal_opt_generate_keys(p, q, g):
17     x = random.randint(1, q - 1) # exposant COURT
18     return x, pow(g, x, p)
19
20 def elgamal_opt_encrypt(p, q, g, y, message):
21     k = random.randint(1, q - 1) # ephemere COURT
22     c1 = pow(g, k, p)
23     c2 = (message * pow(y, k, p)) % p
24     return c1, c2
25
26 def elgamal_opt_decrypt(p, x, c1, c2):
27     s = pow(c1, x, p)
```

```

28 s_inv = inverse(s, p)
29 return (c2 * s_inv) % p

```

5.4 Docker

Dockerfile :

```

1 FROM python:3.11-slim
2 WORKDIR /app
3 RUN pip install pycryptodome
4 COPY . .
5 CMD ["python", "elgamal_cyclique.py"]

```

Build : `docker build -t crypto-test .`

Exécution : `docker run --cpus="0.5" --memory="256m" crypto-test`

Effet : la limite CPU réduit le temps alloué par seconde (suspension du processus, pas baisse de fréquence). La limite RAM peut déclencher du swap si dépassée.

6 Résultats Python

6.1 Pleine puissance

Taille de p	1024	2048	3072	4096
RSA déchiffr. (ms)	3.76	19.8	61.2	168.7
ElGamal déchiffr. (ms)	1.13	2.81	5.84	11.9
Ratio RSA / ElGamal	3.3×	7.0×	10.5×	14.2×

Interprétation. À 4096 bits, ElGamal optimisé est $\sim 14\times$ plus rapide qu’RSA en déchiffrement. Cohérent avec l’analyse théorique : $|p|/|q| = 4096/256 = 16$.

6.2 Sous contrainte Docker

Configuration	RSA déchiffr. 4096 (ms)	ElGamal déchiffr. 4096 (ms)	Ratio
Pleine puissance	169	12	14×
256 Mo / 50 % CPU	230	57	4×
128 Mo / 20 % CPU	610	90	7×

Robuste : dans toutes les conditions Docker testées, ElGamal optimisé est plus rapide qu’RSA en déchiffrement. La structure cyclique apporte un avantage qui résiste aux contraintes matérielles.

6.3 Pourquoi le ratio n’est pas monotone ?

Mesures non moyennées (1 exécution par taille). À 256/50 le ratio est 4×, à 128/20 il est 7× : on s’attendrait à $7\times < 4\times$ par effet de contraction de la ressource, mais le bruit domine. Les tendances qualitatives (toujours ElGamal devant) sont fiables.

6.4 Le chiffrement

RSA reste imbattable en chiffrement, grâce à $e = 65\,537 = 2^{16} + 1$: seulement 17 bits, donc 17 carrés + 1 multiplication. ElGamal fait deux exponentiations avec exposants courts (256 bits) : sous 25 ms à 4096 bits, mais reste plus lent qu’RSA.

7 JavaCard : architecture et implémentation

7.1 Architecture

- Microcontrôleur 16/32 bits, quelques MHz
- ROM (OS), EEPROM (code + données persistantes, écriture ~ 5 ms/octet), RAM transiente (256 octets à quelques Ko)
- **Coprocasseur cryptographique** dédié, généralement RSA (parfois ECC)

7.2 APDU

Tout dialogue host $\leftrightarrow$ carte se fait par APDU : 5 octets d'en-tête (CLA, INS, P1, P2, Lc) + données pour la commande ; données + 2 octets de statut pour la réponse. Mode étendu (**ExtendedLength**) pour réponses > 255 octets.

7.3 Mon applet RSA

```
1 KeyPair RSA_KeyPair = new KeyPair(KeyPair.ALG_RSA ,
2                                   KeyBuilder.LENGTH_RSA_1024);
3 RSA_KeyPair.genKeyPair();
4 rsaCipher = Cipher.getInstance(Cipher.ALG_RSA_PKCS1 , false);
```

L'appel `rsaCipher.doFinal(...)` déclenche *automatiquement* l'exponentiation matérielle.

7.4 Mon applet ElGamal

ElGamal n'est pas dans l'API. Deux astuces :

Détournement du coprocasseur RSA. On charge :

- « Modulus » : le premier p
- « Exposant » : la valeur souhaitée (k pour g^k , x pour c_1^x)

`doFinal` sur la base g (ou c_1) calcule l'exponentiation via le coprocasseur.

Inverse modulaire par Fermat. $a^{-1} \equiv a^{p-2} \pmod{p}$. Une inversion devient une exponentiation $\rightarrow$ coprocasseur.

7.5 Pourquoi ElGamal reste lent

1. Deux exponentiations (vs une pour RSA).
2. Reconfiguration du coprocasseur entre chaque exponentiation.
3. Gestion logicielle de (c_1, c_2) , multiplications modulaires manuelles.
4. Génération de l'éphémère k à chaque chiffrement (PRNG).

8 Résultats JavaCard

8.1 Mesures (une exécution)

Opération	Temps (ms)	Ratio vs RSA
RSA chiffrement	2,22	$1\times$
RSA déchiffrement	4,56	$1\times$
ElGamal chiffrement	32,2	$\sim 14\times$
ElGamal déchiffrement	293	$\sim 64\times$

8.2 Interprétation

Pourquoi RSA est si rapide : entièrement exécuté par le coprocesseur, avec CRT, Montgomery, parallélisme bit-niveau — typiquement 30 à 100× plus rapide qu’une implémentation logicielle.

Pourquoi ElGamal est si lent : le détournement du coprocesseur fonctionne, mais avec un overhead de configuration, deux exponentiations pleines, plus l’inverse modulaire (qui est en fait *une troisième* exponentiation, s^{p-2}).

8.3 Cohérence avec Python

Les deux protocoles ne se contredisent pas :

- **Python** mesure la *logique algorithmique* en isolant les optimisations. ElGamal avec sous-groupe est plus rapide.
- **JavaCard** mesure l’*intégration matérielle*. RSA bénéficie d’un coprocesseur dédié.

9 Synthèse et conclusion

Réponse à la problématique.

La structure algébrique *influence* les performances — mais le sens de l’avantage dépend de ce que l’on optimise.

- **Python** montre l’avantage *algorithmique* d’ElGamal : la cyclicité permet d’exploiter un sous-groupe d’ordre premier q avec $|q| \ll |p|$, donc des exposants courts $\Rightarrow$ exponentiations rapides.
- **JavaCard** montre l’avantage *matériel* d’RSA : trente ans d’optimisation hardware dédiée par l’industrie de la carte à puce.

Les deux structures ont chacune leur niche d’application, selon la cible.

10 Questions probables du jury et éléments de réponse

Question type : Cyclique vs non cyclique

Q : « Vous dites que $(\mathbb{Z}/n\mathbb{Z})^*$ est non cyclique. C’est important pour RSA ? »

R : « Non, et c’est précisément la nuance que mon étude a affinée. La non-cyclicité est une conséquence du CRT, mais RSA ne l’exploite à aucun moment. La sécurité repose sur la factorisation, et la correction sur le théorème d’Euler, qui est valable dans tout groupe abélien fini — cyclique ou non. La vraie opposition est entre un algorithme qui exploite la *structure cyclique* (ElGamal, via le log discret) et un algorithme qui s’appuie sur une *identité algébrique générique* (RSA, via Euler). »

Question type : Construction de g d’ordre q

Q : « Pourquoi $g = \alpha^h$ donne-t-il bien un élément d’ordre q ? »

R : « Par Fermat, $\alpha^{p-1} \equiv 1 \pmod{p}$. Comme $p-1 = hq$, on a $(\alpha^h)^q = \alpha^{p-1} = 1$. Donc l’ordre de g divise q . q étant premier, l’ordre vaut 1 ou q . Si $g \neq 1$, l’ordre vaut exactement q . »

Question type : Sécurité du sous-groupe court

Q : « Vous utilisez un sous-groupe de 256 bits seulement. Est-ce sûr ? »

R : « Oui, c'est précisément le choix du standard DSA. Pollard- ρ dans un sous-groupe d'ordre q donne une sécurité de $\sqrt{q}$, soit 128 bits pour $|q| = 256$. La sécurité contre l'index calculus dans $(\mathbb{Z}/p\mathbb{Z})^*$ entier est dictée par $|p|$, soit 112 bits pour $|p| = 2048$. La sécurité effective est le minimum, comparable à RSA-2048. »

Question type : Pourquoi $e = 65537$?

Q : « Justifiez le choix de e . »

R : « C'est $F_4 = 2^{16} + 1$, un nombre premier de Fermat. Trois raisons : (i) premier, donc le pgcd avec $\varphi(n)$ est facile à vérifier. (ii) Écriture binaire à seulement deux bits à 1, donc m^e se fait en 17 carrés + 1 multiplication. (iii) Suffisamment grand pour éviter les attaques sur petit exposant. »

Question type : Le ratio Python n'est pas monotone

Q : « Vous avez $14\times$ sans limite, $4\times$ à $256/50$, $7\times$ à $128/20$. On s'attendrait à de la monotonie. »

R : « Mes mesures sont sur une seule exécution par taille de clé, non moyennées. La variance est donc importante, surtout sous contrainte mémoire. Une re-exécution avec moyennage sur 30 répétitions est en cours. Ce qui est fiable, c'est la *tendance qualitative* : dans les trois configurations, ElGamal optimisé est plus rapide qu'RSA en déchiffrement. »

Question type : Pourquoi Docker, pas une vraie machine contrainte ?

Q : « Docker n'est pas un environnement embarqué. »

R : « Exact, et c'est précisément pour cela que j'ai prolongé l'étude sur JavaCard. Docker était une approximation pratique pour faire varier les ressources sur ma propre machine. Cela isole l'effet « contrainte de ressources ». Mais c'est la JavaCard qui est l'environnement contraint réel. »

Question type : Différence entre Python et JavaCard

Q : « Vos deux protocoles donnent des résultats opposés. »

R : « Ils ne sont pas contradictoires : ils mesurent des choses différentes. Python isole la complexité algorithmique *pure*, en utilisant pour les deux algorithmes les mêmes primitives logicielles. ElGamal y gagne grâce à ses exposants courts. JavaCard mesure ce qui se passe en *déploiement réel*, où RSA bénéficie d'une accélération matérielle inaccessible à ElGamal. Les deux mesures sont cohérentes : Python traduit l'avantage algorithmique, JavaCard l'avantage matériel. »

Question type : Une seule mesure JavaCard ?

Q : « Vos résultats JavaCard sont sur une seule exécution. »

R : « Oui, c'est une limite que je reconnais. La variance est probablement faible sur la JavaCard (pas de GC concurrent, pas d'OS multitâche), mais une moyenne sur 30 exécutions est en cours. Les opérations cryptographiques matérielles ont en général une variance très réduite, contrairement à Python. »

Question type : Coût de génération de paramètres ElGamal

Q : « Votre génération de paramètres ElGamal prend 37 secondes à 4096 bits. C'est rédhibitoire ? »

R : « C'est un coût élevé, mais à mettre en perspective : la génération est faite *une fois* pour de nombreuses opérations cryptographiques ultérieures. Sur une JavaCard, on génère les paramètres en usine, jamais dans le cadre d'usage normal. Une motivation supplémentaire pour utiliser des paramètres standardisés (groupes RFC 7919 par exemple). »

Question type : Lien avec ECC

Q : « Pourquoi ne pas avoir directement comparé avec ECC ? »

R : « ECC est l'ouverture naturelle de mon TIPE. Je me suis limité à RSA et ElGamal car ils représentent une opposition algébrique propre (anneau vs groupe cyclique) sur des grandes structures commutatives. ECC est essentiellement ElGamal transposé à un autre groupe cyclique, où le log discret est encore plus difficile. C'est exactement la continuation de mon optimisation. »

Question type : Quantique

Q : « La crypto quantique change tout cela ? »

R : « Oui. L'algorithme de Shor (1994), sur ordinateur quantique suffisant, casse RSA, ElGamal et ECC en temps polynomial. La cryptographie post-quantique se tourne vers d'autres problèmes : réseaux euclidiens (Kyber, retenu par le NIST en 2024), codes correcteurs, isogénies. Ces algorithmes posent à leur tour la question de leur adaptation au chiffrement léger — la boucle est bouclée. »

Question type : Choix de Python

Q : « Pourquoi Python pour comparer des temps ? »

R : « Python est lent et bruyant, oui. Deux raisons : (i) implémentation symétrique facilitée. (ii) Les opérations critiques (`pow`) sont en fait exécutées par GMP en C ; le surcoût Python n'affecte que la couche de contrôle. Approximation grossière, et c'est pour cela que la JavaCard est nécessaire. »

Question type : Sécurité sémantique

Q : « ElGamal est sémantiquement sûr, RSA non ? »

R : « RSA déterministe (sans padding) n'a pas de sécurité sémantique : deux chiffrements du même message donnent le même résultat. C'est pourquoi en pratique on utilise OAEP. ElGamal, lui, a la sécurité sémantique nativement grâce à l'aléa k . C'est un avantage qualitatif d'ElGamal qui ne ressort pas dans mes mesures de performance. »

11 Valeurs à connaître par cœur

Grandeur	Valeur
e standard RSA	$65\,537 = 2^{16} + 1$
Taille RSA NIST 112 bits sécurité	2048 bits
Taille RSA NIST 128 bits sécurité	3072 bits
Taille ECC équivalente 128 bits	256 bits
Sous-groupe DSA $ q $	256 bits
Expansion ElGamal	$\times 2$
Coût SaM	$\Theta(b \cdot M(n))$
Gain ElGamal opt. (4096 bits)	$\sim 14\times$ vs RSA
Ratios JavaCard ElGamal/RSA	$\sim 14\times$ enc, $\sim 64\times$ dec

12 Limites à mentionner honnêtement

1. Implémentation Python non optimisée (pas de CRT pour RSA).
2. Mesures Python non moyennées (1 exécution par taille).
3. Une seule exécution JavaCard.
4. Docker n'est pas un vrai environnement embarqué.
5. Pas de test statistique formel.
6. g pas garanti générateur dans *l'ElGamal initial sans sous-groupe* (corrigé dans la version finale).
7. Comparaison limitée à deux algorithmes.